PA4 — A-Frame VR Scene with Custom Component

Sultan Mykhtybayev · Student ID 241280 · prefix `x1280-` · A-Frame 1.8.0 (Three.js r184) · Chrome, laptop · Repository: github.com/kakebun/PA4_ARVRXR

Digit	Rule	Variant
last = 0	theme	Space station
8	8 mod 3 = 2	pulse motion
2	2 mod 2 = 0	gaze, fuse cursor 1.5 s
1	1 mod 2 = 1	corridor: 2 rows × 3, rows 3 m apart, z = −2 … −12

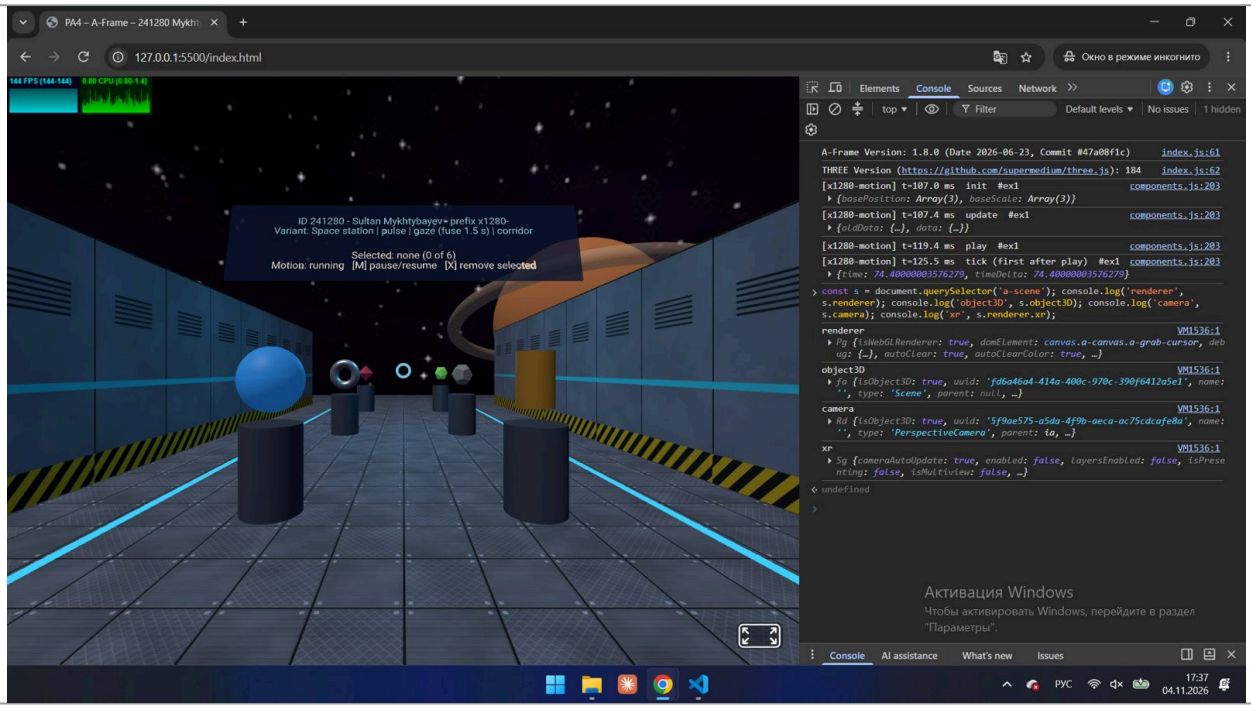


Fig. 0 (screenshot 9) — The scene with the ID/variant panel and the live counter “Selected: none (0 of 6)”. 144 fps on the laptop (stats).

E1 — The component life cycle

Prediction On load: `init → update → play → tick`. `setAttribute` calls only `update`. `el.pause()/play()` call the component’s `pause/play`. `removeAttribute` calls `remove`. Removing the entity calls `remove` again.

Result `#ex1` has `log: true`; each method prints `performance.now()`.

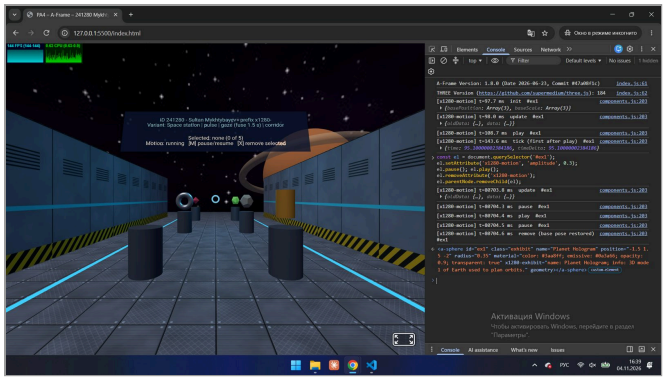


Fig. 1a (screenshot 1) — Full sequence: load (t≈98 ms) and the console commands (t≈80 704 ms).

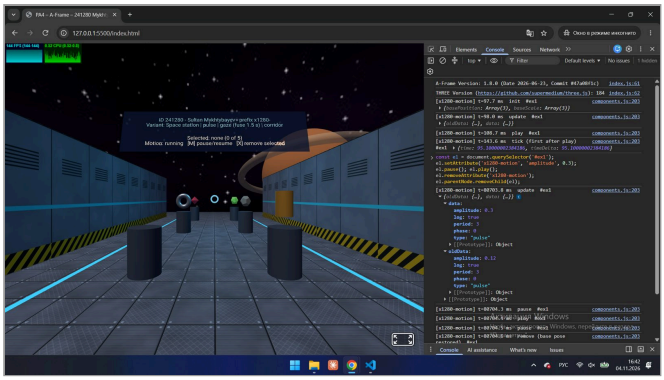


Fig. 1b (screenshot 2) — Second update expanded: `oldData.amplitude = 0.12`, `data.amplitude = 0.3`.

Order: `init → update → play → tick`, then `update → pause → play → pause → remove`; removing the entity printed nothing.

Explanation `update` first fires right after `init` with `oldData = {}` (my code uses this to detect the first call), then whenever parsed data changes; `oldData` is a copy of the previous data (`amplitude: 0.12`). Two predictions were wrong: `removeAttribute` calls `pause` before `remove`, and removing the entity logs nothing because the component no longer exists. `tick` runs only while playing.

E2 — Checking Ch. 8

Prediction (a) Runtime objects declared in the schema will be treated as configuration: copied, compared and reset to defaults.

Result Ch. 8 structure (runtime objects in the schema, assigned in `init`), then `setAttribute` on one property and a full replace:

```
before:  {src: "textures/deck.jpg", repeat: {x:2,y:2}, texture: Texture, material:
MeshStandardMaterial, mesh: Mesh}
setAttribute('repeat', {x:10,y:10})  → data.repeat = {x:10,y:10}, same texture instance
in update(): oldData.texture === data.texture → false   (A-Frame copied the Texture into
oldData)
setAttribute('…', 'src: textures/panel.jpg; repeat: 3 3', true) → texture: null, material: null,
mesh: null
Object.keys(sceneEl.systems): camera, geometry, gltf-model, light, material, obb-collider,
renderer,
                              shadow, tracked-controls, webxr, x1280-gallery   (no "assets")
```

Explanation (a) `this.data` is rebuilt from the attribute plus defaults, so runtime objects in it are copied into `oldData` on every update and wiped to `null` when the attribute is replaced, which leaks the undisposed texture and material. The schema should hold only what can be typed in HTML; runtime objects belong on `this`, as in `x1280-texture`.

(b) `<a-assets>` is a custom DOM element (`customElements.define('a-assets', AAssets)`, a-assets.js). that preloads media and delays scene start. `registerSystem` creates a scene-level singleton at `sceneEl.systems[name]` with `init/tick/pause/play`, shared by component instances. `assets` is absent from `sceneEl.systems`, so the claim does not hold.

E3 — Texture filtering

Prediction Nearest without mipmaps will shimmer, the default will blur far tiles, anisotropy will keep them sharp.

Result Camera at 0.3 m (`#rig position 0 −1.3 2`), same view, `x1280SetFilters()`:

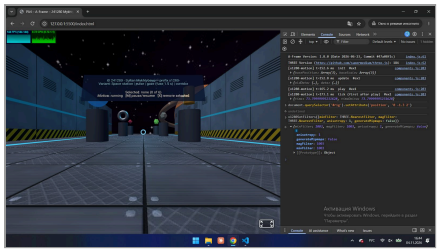


Fig. 3a (screenshot 4) — Ch. 8 filters: Nearest/Nearest (1003), no mipmaps.

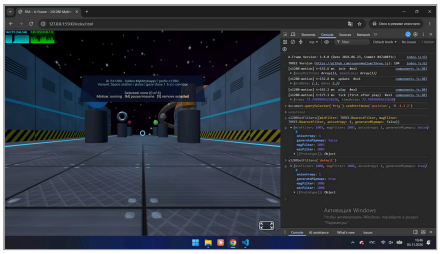


Fig. 3b (screenshot 5) — Three.js defaults: LinearMipmapLinear (1008) / Linear (1006), anisotropy 1.

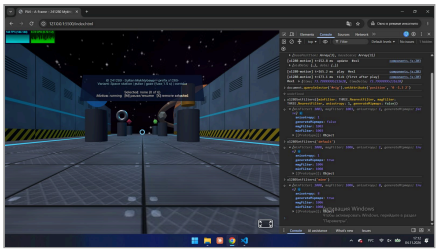


Fig. 3c (screenshot 6) — Mine: 1008 / 1006, anisotropy 8.



Fig. 3d — Same floor region cropped from screenshots 4–6: aliased and noisy (top), blurred (middle), sharp and stable (bottom).

Explanation **Magnification**: one texel covers many pixels (near); Linear interpolates, Nearest is blocky. **Minification**: many texels per pixel (far); one sample picks a different texel each frame, the shimmer in Fig. 3a. **Mipmaps** are pre-filtered half-size copies; trilinear blends two levels, but at grazing angles the footprint is long and thin, so isotropic selection is too blurry (Fig. 3b). **Anisotropy** samples along the footprint (Fig. 3c). My choice — trilinear + linear + anisotropy 8, clamped to `getMaxAnisotropy()` — is cheap and removes shimmer, which is worse in VR. Property check: `x1280SetFilters` warns for any key missing on `THREE.Texture`; `minFilter`, `magFilter`, `anisotropy`, `generateMipmaps`, `wrapS/T`, `repeat`, `colorSpace` exist — unknown names are silently ignored by Three.js.

E4 — What <a-scene> builds

Prediction `<a-scene>` wraps the same Three.js objects I created by hand in PA3.

Result

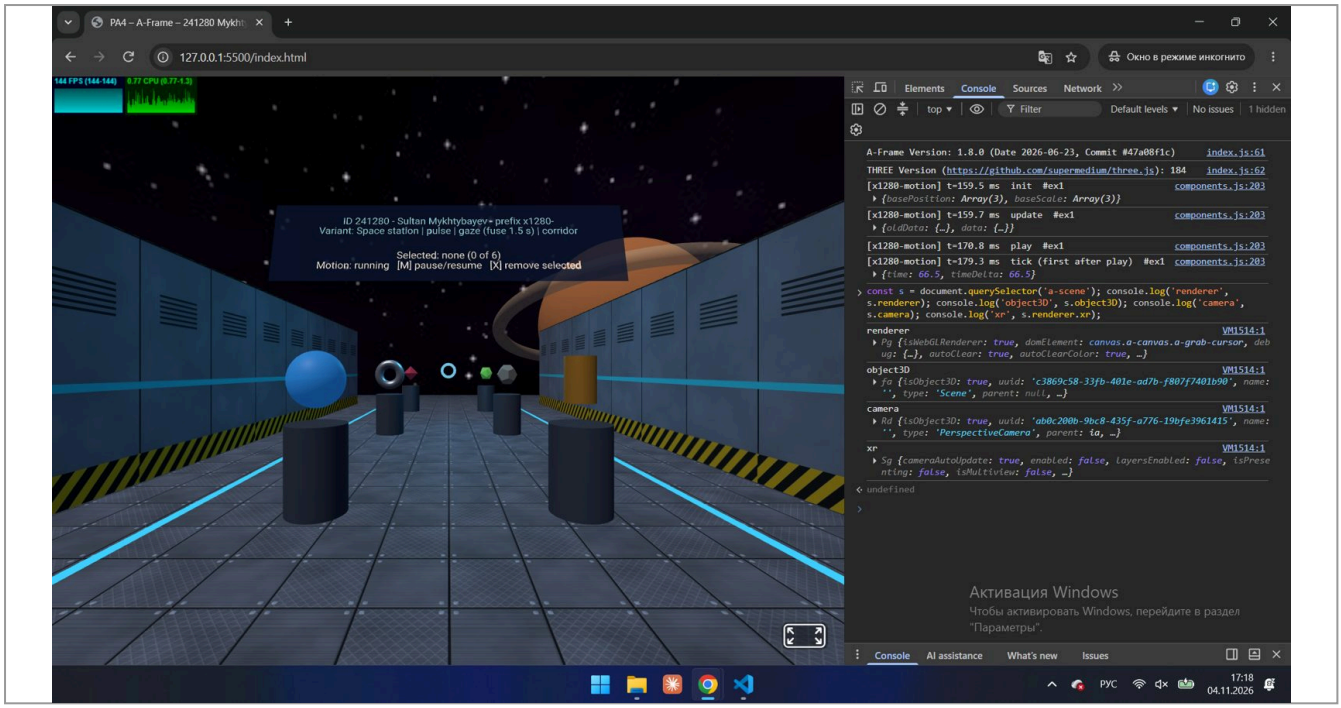


Fig. 4 (screenshot 8) — `sceneEl.renderer` = WebGLRenderer, `sceneEl.object3D` = Scene, `sceneEl.camera` = PerspectiveCamera, `renderer.xr` = WebXRManager (enabled: false, isPresenting: false before Enter VR).

The session starts in `AScene.enterVR()` (A-Frame 1.8.0, a-scene.js):

```
vrManager.enabled = true;
var xrMode = useAR ? 'immersive-ar' : 'immersive-vr';
xrInit = this.sceneEl.systems.webxr.sessionConfiguration;
navigator.xr.requestSession(xrMode, xrInit).then(function (xrSession) {
  vrManager.setSession(xrSession).then(function () { ... enterVRSuccess(resolve); });
});
```

Explanation In PA3 I wrote this chain myself (check `navigator.xr`, `requestSession` on a click, `renderer.xr.setSession()`, `setAnimationLoop()`, handle `end`); A-Frame adds reference space, frame rate, foveation and exit. **Raw Three.js** suits a company needing full pipeline control or a small bundle, e.g. a configurator inside an existing React/Three.js app with custom post-processing. **A-Frame** suits fast prototypes by web teams, e.g. an exhibition or training scene for Quest and desktop with reusable components.

E5 — Performance

Prediction Each extra exhibit adds about one draw call. 60 instanced copies add only one.

Result Read from `renderer.info.render`. The compact stats panel of A-Frame 1.8 shows fps only.

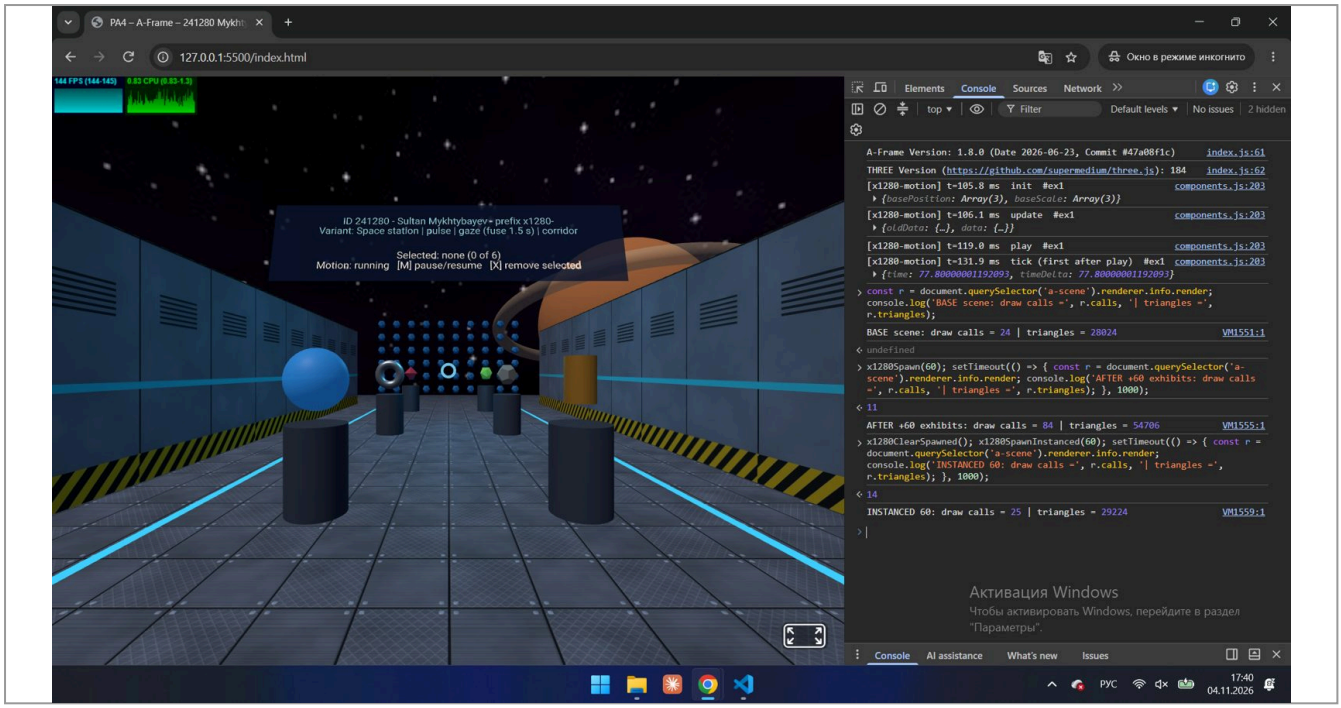


Fig. 5 (screenshot 10) — Console values for the three runs; fps stayed at 144 (display limit) in all of them.

Scene	Draw calls	Triangles	fps (laptop)
Base scene	24	28 024	144
+60 exhibits (`x1280Spawn(60)`)	84	54 706	144
60 objects as one InstancedMesh (`x1280SpawnInstanced(60)`)	25	29 224	144

Explanation Each draw call costs CPU time (state validation, material binding, uniforms). VR renders two eyes — without multiview, twice the calls — within 13.9 ms at 72 Hz on a mobile CPU; missing it causes judder and sickness. A laptop at 144 fps hides 84 calls, a Quest would not; 54 k triangles are cheap. One shared geometry and material in a `THREE.InstancedMesh` turned 60 objects into one call (84 → 25).

E6 — Comfort and design

Exhibits stand 1.5 m either side of the walking line at y = 1.5 m (eye 1.6 m): readable, outside arm's reach, no neck tilt; groups are 5 m apart. Board glyphs are ≈ 0.043 m (2.5 m / 58 chars) at 2.77 m ≈ 0.9°; exhibit panels 0.044 m at ≈ 2 m ≈ 1.3° — near the 1° minimum for VR text. The 1.5 s fuse avoids accidental selection while looking around; the shrinking ring shows progress; the raycaster only hits `.exhibit`. **Comfort risk:** smooth WASD movement in VR causes vection sickness. I reduced acceleration from 65 to 20, attached the board to the rig instead of the head, and exhibits pulse in place rather than travel.

Development log

Bug: the first GitHub web upload placed `deck.jpg` and the other textures in the repository root, so `index.html` would get 404s for `textures/…` and the floor and walls would lose their textures.
Found it: by checking the upload list before committing. The paths had no `/textures/` prefix, while the earlier upload showed `/textures/deck.jpg`.
Fix: I uploaded the `textures` and `tools` folders by drag-and-drop so the structure was kept. Commit `61b29cd` "Add README, textures and generator script".